

Module 3 — Spring Boot

Spring Boot = Spring + opinionated auto-configuration + embedded server + production-ready features. Interviewers want to know *how the magic works* (auto-config, starters, the embedded container) so you can debug it.

3.1 Auto-Configuration

Why Interviewers Ask This

It's *the* Spring Boot differentiator. If you can explain how Boot configures a `DataSource` you never declared, you understand the framework, not just the demo.

Core Concept

Auto-configuration conditionally registers beans based on what's on the classpath, existing beans, and properties — so sensible defaults appear automatically, yet you can always override them.

Internal Working

- `@SpringBootApplication` = `@SpringBootConfiguration` + `@ComponentScan` + `@EnableAutoConfiguration`.
- `@EnableAutoConfiguration` imports `AutoConfigurationImportSelector`, which reads candidate config classes from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (pre-2.7: `spring.factories`).
- Each candidate is guarded by `@Conditional` annotations:
- `@ConditionalOnClass` / `@ConditionalOnMissingClass`
- `@ConditionalOnBean` / `@ConditionalOnMissingBean` (your bean wins!)
- `@ConditionalOnProperty`
- `@ConditionalOnWebApplication`
- Ordering via `@AutoConfigureBefore/After/Order`.

ASCII — Auto-config Flow

```
@SpringBootApplication
-> @EnableAutoConfiguration
    -> AutoConfigurationImportSelector
        -> read AutoConfiguration.imports (100s of candidates)
            -> for each: evaluate @Conditional*
                pass? register beans (DataSource, DispatcherServlet, ...)
                user already defined bean? @ConditionalOnMissingBean -> skip
```

Real Production Example

Add `spring-boot-starter-data-jpa` + an H2 dependency → Boot auto-creates a `DataSource`, `EntityManagerFactory`, `JpaTransactionManager`, and configures Hibernate — zero XML. Define your own `DataSource` bean and Boot backs off (`@ConditionalOnMissingBean`).

Advantages / Trade-offs

- Massive boilerplate reduction, consistent defaults, override-friendly. – “Magic” can be opaque; classpath changes silently alter behavior; startup does a lot of conditional evaluation.

Common Mistakes / Debugging

Assuming a bean is missing when auto-config backed off (or vice versa). Use the **condition evaluation report**: run with `--debug` (or `/actuator/conditions`) to see *why* each auto-config matched or not. Exclude with `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.

Interview Q / Follow-ups

- How does auto-configuration work end to end?
- What is `@ConditionalOnMissingBean` and why does it matter for overriding?
- Where are auto-config classes registered? (`AutoConfiguration.imports`.)
- How do you debug why a bean was/wasn't created? (`--debug conditions report`.)
- How do you write your own starter/auto-config?

Hands-on Exercise

Write a custom auto-configuration class guarded by `@ConditionalOnProperty` and `@ConditionalOnMissingBean`, register it in `AutoConfiguration.imports`, and verify it via the conditions report.

Best Practices

Prefer properties over disabling auto-config; use `@ConditionalOnMissingBean` in your own starters; keep the classpath intentional.

3.2 Starter Dependencies

Core Concept

Starter are curated, versioned dependency bundles (BOM-managed) — e.g. `spring-boot-starter-web` pulls Spring MVC, Jackson, validation, embedded Tomcat. The **parent BOM** (`spring-boot-dependencies`) fixes compatible versions so you don't manage them individually.

Interview Q

- What does a starter actually contain? (*transitive deps + version alignment, no code*.)
- How does Boot manage versions? (*dependency management BOM*.)
- Common starters: `-web`, `-data-jpa`, `-security`, `-actuator`, `-validation`, `-webflux`, `-test`.

3.3 Embedded Server (Tomcat)

Core Concept

Boot embeds the servlet container (default **Tomcat**; alternatives Jetty, Undertow) *inside* the fat JAR. `java -jar app.jar` starts an HTTP server — no external app server / WAR deployment.

Internal Working & Lifecycle

`SpringApplication.run()` → creates `ServletWebServerApplicationContext` → `ServletWebServerFactory` (Tomcat) starts the connector → registers the `DispatcherServlet` → binds the port. Graceful shutdown drains in-flight requests (`server.shutdown=graceful`).

ASCII

```
java -jar app.jar
-> SpringApplication.run()
    -> create ApplicationContext (servlet-aware)
    -> TomcatServletWebServerFactory.getWebServer().start()
    -> register DispatcherServlet, filters
    -> listen on :8080
```

Trade-offs / Tuning

Tune `server.tomcat.threads.max` (default 200), `accept-count`, `max-connections`, `connection-timeout`. Undertow is lightweight; WebFlux/Netty for reactive. Thread-per-request means blocking calls consume worker threads.

Interview Q / Follow-ups

- Embedded vs external server — pros/cons? (*self-contained deploys, versioned per app vs shared infra.*)
- How to switch to Undertow/Jetty? (*exclude Tomcat starter, add the other.*)
- What is the default Tomcat thread pool size and how does it affect throughput?

3.4 Configuration Properties, Profiles & Logging

@ConfigurationProperties

Type-safe binding of a property group to a POJO/record; supports validation (`@Validated` + JSR-380), nested objects, lists, `Duration/DataSize`, and relaxed binding (`app.max-size` ↔ `APP_MAXSIZE`).

```
@ConfigurationProperties(prefix = "app.rate-limit")
public record RateLimitProps(int perMinute, Duration window) {}
```

Profiles

`application-{profile}.yml` + `spring.profiles.active`. Profile groups (`spring.profiles.group.prod=db,cache`). `@Profile` on beans.

Logging

Boot uses **SLF4J + Logback** by default. Configure levels via `logging.level.com.acme=DEBUG`; use `logback-spring.xml` for appenders/JSON encoding. Structured (JSON) logs for centralized logging; MDC for correlation IDs.

Interview Q

- `@Value` vs `@ConfigurationProperties` (type safety, grouping, validation).
- How to log JSON with a trace/correlation id? (*logback JSON encoder + MDC.*)

3.5 Validation

Core Concept

Bean Validation (JSR-380 / Jakarta Validation, Hibernate Validator impl). Annotate DTO fields (`@NotNull`, `@NotBlank`, `@Email`, `@Size`, `@Min`, `@Pattern`); trigger with `@Valid`/`@Validated` on controller params. Group validation and custom `ConstraintValidator` for domain rules.

```
public record CreateUser(@NotBlank String name,
                        @Email String email,
                        @Min(18) int age) {}

@PostMapping("/users")
User create(@Valid @RequestBody CreateUser dto) { ... }
```

Failures throw `MethodArgumentNotValidException` → handle globally (3.6).

Common Mistakes

Forgetting `@Valid`; validating entities instead of DTOs; `@Validated` (class level, needed for method-parameter validation on `@Service`) vs `@Valid`.

3.6 Exception Handling

Core Concept

Centralize error handling with `@RestControllerAdvice` + `@ExceptionHandler`, returning a consistent error body (ideally [RFC 7807 ProblemDetail](#), built-in since Spring 6).

```
@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail onValidation(MethodArgumentNotValidException ex) {
        var pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setTitle("Validation failed");
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors()
            .stream().collect(toMap(FieldError::getField, FieldError::getDefaultMessage)));
        return pd;
    }
    @ExceptionHandler(EntityNotFoundException.class)
    ProblemDetail onNotFound(EntityNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    }
}
```

Interview Q / Follow-ups

- `@ControllerAdvice` vs `@RestControllerAdvice`? (latter adds `@ResponseBody`.)
- How to return consistent error responses? (`ProblemDetail`.)
- Ordering multiple advices? `@Order`. Handling `ResponseStatusException`?

3.7 REST API Best Practices

- **Nouns + plurals:** `/orders/{id}/items`; no verbs in paths.
- **HTTP methods:** GET (safe/idempotent), POST (create), PUT (full replace, idempotent), PATCH (partial), DELETE (idempotent).
- **Status codes:** 200/201/204, 400/401/403/404/409/422, 429, 500/503.
- **Versioning:** URI (`/v1`), header, or media-type.
- **Pagination/filtering/sorting;** **HATEOAS** where useful.
- **Idempotency keys** for POST retries; **consistent error schema;** **DTOs** (never expose entities).

3.8 Filters vs Interceptors

	Filter (Servlet)	Interceptor (Spring MVC)
Layer	Servlet container, before DispatcherServlet	Inside MVC, around handler
API	<code>jakarta.servlet.Filter</code>	<code>HandlerInterceptor</code>
Access	raw request/response	handler, ModelAndView
Use	auth, CORS, gzip, logging, correlation id	auth on handlers, timing, MDC
Order	<code>@Order/FilterRegistrationBean</code>	registration order

ASCII — Where they sit

```
Client -> [ Servlet Filters ] -> DispatcherServlet -> [ Interceptors.preHandle ]
        -> Controller -> [ Interceptors.postHandle ] -> view/body
        -> [ Interceptors.afterCompletion ] -> [ Filters (unwind) ] -> Client
```

Interview Q

Filter vs interceptor vs AOP; where to set a correlation id (filter, early); `OncePerRequestFilter` and why.

3.9 Actuator

Core Concept

Production-ready endpoints under `/actuator`: `/health` (liveness/readiness groups), `/info`, `/metrics`, `/prometheus`, `/env`, `/loggers` (change log levels at runtime!), `/threaddump`, `/heapdump`, `/httpexchanges`, `/mappings`, `/conditions`.

Best Practices / Security

Expose only what you need (`management.endpoints.web.exposure.include=health,info,prometheus`); secure actuator (separate port/auth); use health groups for K8s liveness vs readiness probes.

Interview Q

- Difference between liveness and readiness probes and how actuator maps to them.
- How to change log level without redeploy? (`/actuator/loggers` POST.)

Module 3 — One-Page Cheat Sheet

Topic	Key point
@SpringBootApplication	=Config + ComponentScan + EnableAutoConfiguration
Auto-config	conditional beans from <code>AutoConfiguration.imports</code> ; <code>@ConditionalOnMissingBean</code> lets you override
Debug config	<code>--debug</code> / <code>/actuator/conditions</code> report
Starters	curated deps + BOM version alignment
Embedded Tomcat	fat JAR, <code>java -jar</code> ; ~200 worker threads
@ConfigurationProperties	type-safe, validated, relaxed binding
Validation	<code>@Valid</code> DTOs → <code>MethodArgumentNotValidException</code>
Errors	<code>@RestControllerAdvice</code> + <code>ProblemDetail</code> (RFC 7807)
Filter vs Interceptor	filter=servlet layer; interceptor=MVC layer
Actuator	health/metrics/loggers/prometheus; secure it

Module 3 — Top Interview Questions

1. Explain auto-configuration internals (`@Conditional`, imports file, back-off).
2. What does `@SpringBootApplication` combine?
3. How to override an auto-configured bean?
4. Embedded vs external server; tune the Tomcat thread pool.
5. `@Value` vs `@ConfigurationProperties`.
6. Global exception handling & consistent error responses.
7. Filter vs interceptor — when to use each.
8. Actuator: liveness vs readiness; runtime log level change.
9. How would you build a custom starter?
10. REST best practices (status codes, idempotency, versioning).

Module 3 — Common Mistakes

- Fighting auto-config instead of using properties/back-off.
- Exposing all actuator endpoints publicly.
- Forgetting `@Valid`; exposing entities instead of DTOs.
- Blocking calls saturating the fixed Tomcat thread pool.
- Not using health groups for K8s probes.

Module 3 — Mock Interview

1. “Boot created a `DataSource` I didn’t declare — how, and how do I override it?” → auto-config via `@ConditionalOnClass` + `@ConditionalOnMissingBean`; declare your own bean and it backs off.
2. “How do I know why bean X wasn’t created?” → conditions report (`--debug`).
3. “Requests queue under load though CPU is idle.” → thread-per-request Tomcat pool exhausted by blocking downstream calls; raise pool / add timeouts / go reactive.
4. “Standardize error responses across 40 endpoints.” → `@RestControllerAdvice` + `ProblemDetail`.

5. "Change log level in prod without redeploy." → `POST /actuator/loggers/com.acme {"configuredLevel":"DEBUG"}`.

Next → Module 4: Spring MVC (DispatcherServlet & the request lifecycle).